

Summary of changes: pCN sampler for `spinDMFT`

1 Overview

The original `spinDMFT/Algorithm/` samples the mean-field trajectory V from the Gaussian prior $p(V)$ and reweights observables by the per-trajectory partition function $Z(V) = \text{Tr}[U(\beta, 0; V)]$. At low temperatures $Z(V)$ becomes sharply peaked in the prior support, leaving the importance-sampling estimator weight-degenerate and forcing extremely large sample counts.

The new directory `spinDMFT/Algorithm.MH/` is a self-contained copy of the algorithm that replaces importance sampling with a preconditioned Crank–Nicolson (pCN) Markov chain [1] targeting

$$\pi(dV) \propto p(V) Z(V) dV, \quad (1)$$

the measure under which the sample mean of $\text{Tr}[U(\beta-\tau) S^\alpha U(\tau) S^\beta]/Z(V)$ is already the thermal correlation $g^{\alpha\beta}(\tau)$. No reweighting is needed; for Gaussian-prior, positive-likelihood targets pCN has dimension-independent autocorrelation, so the effective sample size is on par with that of independent draws from the target.

2 File-by-file changes

2.1 `Parameter_Space/`

Three new CLI options are exposed:

- `--mhStepSize` (`RealType`, default 0.3). The pCN step parameter $\beta_{\text{pCN}} \in (0, 1]$. Larger β_{pCN} drives the proposal closer to an independent draw from the prior; smaller β_{pCN} takes more local moves.
- `--mhBurnIn` (`size_t`, default 100). Number of cold-start warm-up pCN steps per MPI core, used in the first self-consistent iteration and discarded before observables are accumulated.
- `--mhWarmBurnIn` (`RealType`, default 0.25). Fraction of `mhBurnIn` used for the warm-start refresh in every self-consistent iteration after the first. At iteration $k > 1$ the chain is seeded with the previous iteration’s final V rotated into the new diagonal basis, so a fraction of the cold burn-in length is typically sufficient. Set to 0 to skip the refresh entirely (maximum speedup, but watch for residual bias when early-iteration acceptance is low); set to 1 to recover the old per-iteration cold burn-in cost.

The corresponding members `ParameterSpace::mh_step_size`, `ParameterSpace::mh_burn_in`, and `ParameterSpace::mh_warm_burn_in_frac` are added to the header.

2.2 Run_Time_Data/

A small acceptance-rate diagnostic is added. Per MPI core, the chain counts proposed and accepted moves into `mh_proposed_count` and `mh_accepted_count`; at the end of each self-consistent iteration the new method `record_mh_acceptance()` performs an `MPI_Allreduce` (`MPI_SUM`) and appends the global ratio to the member `acceptance_rates`, which is also printed at iteration end and persisted to the HDF5 output (§4).

Several IS-only accumulators are removed since the MH chain produces unweighted samples from π : `sample_cov_Re`, `sample_cov_Im`, `spin_expval_cov`, and `Z_sqsum` are gone. The standard-error computation in `compute_sample_stds` is correspondingly rewritten as the textbook formula

$$\text{stderr}(\langle o \rangle) = \sqrt{(\langle o^2 \rangle - \langle o \rangle^2)/N}, \quad (2)$$

where $\langle o \rangle$ is the (already normalized) Monte-Carlo mean, $\langle o^2 \rangle = \sum_i o_i^2/N$ comes from the running sum-of-squares `sample_sqsum_*`, and N is the total number of MC samples across all cores. Both correlations and spin expectation values share this single helper.

2.3 Functions/Functions.h and Functions.cpp

Two new pieces live in namespace `spinDMFT::Functions`:

- `diag_freq_to_time(V_diag, ortho)` – given the chain state expressed in the diagonal basis of the per-block prior covariance (eigenvalues $\lambda_{n,\alpha}$), returns the corresponding imaginary-time trajectory $V(\tau_n)$. Mirrors the back-transform conventions of `fmvg::FrequencyNoiseVectors::fourier_back_tr`
- Class `PCNChain` – the entire pCN chain, described in §2.4.

The single most important change to the existing helpers is to `compute_spin_correlations`. The original IS estimator accumulates the unweighted trace $\text{Tr}[U(\beta-\tau) S^\alpha U(\tau) S^\beta]$ and the partition function Z separately, dividing at the end. Under MH sampling from π , the per-sample observable is already the correlation

$$o^{\alpha\beta}(\tau; V) = \frac{\text{Tr}[U(\beta-\tau; V) S^\alpha U(\tau; V) S^\beta]}{Z(V)}, \quad (3)$$

so each per-sample contribution is divided by $Z(V)$ inside the accumulator. The function now reads (schematically)

```
const RealType inv_Z = ( Z != 0 ) ? 1. / Z : 0.;
const RealType o      = std::real( single_sample_result ) * inv_Z;
spin_correlations    += o;      // sum of o_i, divided by N at the end -> <o>
sample_sqsum         += o * o; // sum of o_i^2, used in stderr formula
```

Because the chain produces unweighted samples from π , there is no IS covariance term: `MPI_share_results` and `compute_sample_stds` drop the `sample_cov_*` and `Z_sqsum` machinery entirely (see the `Run_Time_Data/` section). The global “partition function” passed to `normalize` is replaced by the total sample count $N = \text{my_pspace.num_Samples}$. Several dead parameters were also dropped along the way: `compute_spin_correlations` no longer takes `pspace`, and `normalize` no longer takes `rtdata`. The MPI helper `MPI_share_results` grew a small private utility `mpi_sum_fieldvector` that calls `MPI_Allreduce` directly on the contiguous storage of `FieldVector`, removing a few rounds of buffer copying.

2.4 PCNChain class (lives in Functions/)

All of the chain machinery is encapsulated in a single class whose only public surface is a constructor, a `step()` method, and a handful of accessors.

- **State held by the chain.** Diagonal-basis position V_{diag} , current $Z(V)$, the cached propagators $U(\tau)$ and $U(\tau)^{-1}$, and the cached spin trajectories $S^\alpha(\tau)$. Also held: a per-mode `std::normal_distribution` for proposal noise, and a `Uniform(0, 1)` for accept/reject.
- **Cold-start constructor.** Builds the per-mode prior distributions from the (possibly truncated) eigenvalues, draws an initial V_{diag} from the prior, and retries up to 32 times if $Z(V) \leq 0$.
- **Warm-start constructor.** Same construction, but seeds V_{diag} from a caller-supplied vector instead of the prior draw. Components in modes that the new prior has frozen ($\lambda \leq 0$) are forced to zero. Used to carry chain state across self-consistent iterations (§2.5).
- **step().** Proposes $V'_{\text{diag}} = \sqrt{1 - \beta_{\text{pCN}}^2} V_{\text{diag}} + \beta_{\text{pCN}} \xi$ with $\xi \sim p$, computes $Z(V')$, accepts with probability $\min(1, Z'/Z)$ ($Z \leq 0$ treated as forbidden), and on acceptance refreshes the cached propagators and $S^\alpha(\tau)$.
- **Accessors.** `Z()`, `S_x()`, `S_y()`, `S_z()` for the observable accumulator; `V_full()` returns the chain state in the basis-independent Matsubara representation $V_{\text{full}}[b] = O[b] V_{\text{diag}}[b]$ for cross-iteration state transfer.

2.5 main.cpp

The Monte-Carlo loop becomes a thin shell around `PCNChain`. Across the self-consistent iterations the loop carries a per-rank `std::vector<fmvg::Vector> my_V_full_persistent` that is empty before the first iteration and holds the previous iteration's final V_{full} thereafter.

1. **Chain construction.** If `my_V_full_persistent` is empty (first iteration), build the chain with the cold-start constructor. Otherwise rotate the saved V_{full} into the new diagonal basis,

$$V_{\text{diag,new}}[b] = O_{\text{new}}[b]^\top V_{\text{full}}[b], \quad (4)$$

and pass it to the warm-start constructor.

2. **Burn-in.** In the first iteration, run `mhBurnIn` pCN steps. In every subsequent iteration, run `[mhWarmBurnIn · mhBurnIn]` refresh steps. Acceptance counters update in both phases for diagnostics.
3. **Production.** `numSamplesPerCore` pCN steps; each step calls `chain.step()` and feeds the (cached or newly rebuilt) `(chain.Z(), chain.S_x(), ...)` to `compute_spin_correlations`. On a rejected step the same trajectory is re-accumulated, which correctly weights the current state by its holding time.
4. **State capture.** After the production sweep, `my_V_full_persistent = chain.V_full()` stores the chain state in the basis-independent representation, ready for the next iteration's warm rotation.
5. **Normalization plumbing.** The IS variable `my_partition_function` is gone; in its place `N = my_pspace.numSamples` is passed directly to `normalize` and `compute_sample_stds`. After `record_mh_acceptance()`, the iteration finishes through the existing convergence check and storage path unchanged.

Empirical note on warm-start. When early-iteration acceptance is low ($\lesssim 20\%$, as happens at low temperature before the self-consistent covariance settles), a small `mhWarmBurnIn` fraction can carry residual bias from the initial chain state into the final estimator. At $\beta J = 12$, `mhBurnIn` = 2000, `mhWarmBurnIn` = 0 gives a $\sim 5\sigma$ bias relative to the cold-only result; `mhWarmBurnIn` = 1 recovers it. The default of 0.25 is a safe choice for high-temperature sweeps; raise it towards 1 in deep-cold regimes.

2.6 CMakeLists.txt

The output binary is renamed from `executable.DOUBLE.out` to `executable.MH.DOUBLE.out` so the pCN binary and the original IS binary can coexist in `spinDMFT/`.

3 Mathematical contract of the new estimator

The original IS code computes

$$\hat{g}_{\text{IS}}^{\alpha\beta}(\tau) = \frac{\sum_{i=1}^N \text{Tr}[U(\beta-\tau; V_i) S^\alpha U(\tau; V_i) S^\beta]}{\sum_{i=1}^N Z(V_i)}, \quad V_i \sim p(V). \quad (5)$$

The new code computes

$$\hat{g}_{\text{pCN}}^{\alpha\beta}(\tau) = \frac{1}{N} \sum_{i=1}^N \frac{\text{Tr}[U(\beta-\tau; V_i) S^\alpha U(\tau; V_i) S^\beta]}{Z(V_i)}, \quad V_i \sim \pi. \quad (6)$$

Both target the same thermal expectation $g^{\alpha\beta}(\tau)$. The pCN form avoids the IS denominator, which is the source of the weight-degeneracy problem at low temperature: when $Z(V)$ is sharply peaked the IS estimator concentrates all its statistical weight on a vanishing fraction of the prior samples, whereas the pCN chain draws preferentially from the peaked region and weights every sample equally.

4 Diagnostic interface

Every self-consistent iteration now prints

MH acceptance rate : <number>

after the standard convergence-error report. The full per-iteration time series is held in `RunTimeData::acceptance` during the run and persisted to the HDF5 output as the attribute `runtimedata/mh_acceptance_rates` (a list with one entry per completed self-consistent iteration), alongside the existing `absolute_iteration_errors` and `negative_eigenvalue_ratios`. A practical tuning target is 30–60% acceptance; below $\sim 20\%$ shrink `--mhStepSize`, above $\sim 80\%$ enlarge it.

5 Tested regime

Convergence has been verified against Grempel’s reference (`Data/grempel_plot.csv`) at $\beta J \in \{1, 2, 3, 5, 7, 12, 30\}$ with `numTimeSteps`=100 and `cstype`=A. Relative agreement at $\tau = \beta/2$ is everywhere $\leq 1.2\%$. At $\beta J = 30$ the chain converges in ~ 5 self-consistent iterations with `--mhStepSize`=0.3 (acceptance $\sim 50\%$); longer chains ($\sim 8 \times 10^5$ total samples, 8×10^4 burn-in) bring the $\tau \leftrightarrow \beta - \tau$ asymmetry below 4×10^{-4} .

References

- [1] S. L. Cotter, G. O. Roberts, A. M. Stuart, D. White, *MCMC methods for functions: modifying old algorithms to make them faster*, Statistical Science **28**, 424 (2013).